Name – Deepanshu

Roll no – 2401010133

Section – C

Assignment no – 1

Subject – ADA LAB

Link - https://github.com/001-Deepanshu/ADA-LAB-ASSIGNMENT-1

hello.py > linear_time

```python
import time
import random
import matplotlib.pyplot as plt

def measure_time(func, n, runs=5):
    total = 0
    for _ in range(runs):
        start = time.time()
        func(n)
        total += (time.time() - start)
    return total / runs

def constant_time(n):
    return n * 2

def linear_time(n):
    total = 0
    for i in range(n):
        total += i
    return total

def quadratic_time(n):
    total = 0
    for i in range(n):
        for j in range(n):
            total += i + j
    return total

def logarithmic_time(n):
    count = 0
    while n > 1:
        n = n // 2
        count += 1
    return count


sizes = [10, 100, 500, 1000]

t_const = [measure_time(constant_time, n) for n in sizes]
t_linear = [measure_time(linear_time, n) for n in sizes]
t_quad = [measure_time(quadratic_time, n) for n in sizes]
t_log = [measure_time(logarithmic_time, n) for n in sizes]

plt.figure()
plt.plot(sizes, t_const, label="O(1)")
```

hello.py > linear_time

```python
plt.plot(sizes, t_linear, label="O(n)")
plt.plot(sizes, t_quad, label="O(n^2)")
plt.plot(sizes, t_log, label="O(log n)")
plt.xlabel("Input Size")
plt.ylabel("Time")
plt.title("Algorithm Growth Comparison")
plt.legend()
plt.show()

def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

def binary_search(arr, key):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1

sizes = [100, 500, 1000, 2000]

lin_best, lin_avg, lin_worst = [], [], []
bin_best, bin_avg, bin_worst = [], [], []

for n in sizes:
    arr = sorted(random.sample(range(n * 10), n))

    best = arr[0]
    avg = arr[n // 2]
    worst = -1

    # Linear Search
    lin_best.append(measure_time(lambda x: linear_search(arr, best), n))
    lin_avg.append(measure_time(lambda x: linear_search(arr, avg), n))
    lin_worst.append(measure_time(lambda x: linear_search(arr, worst), n))
```

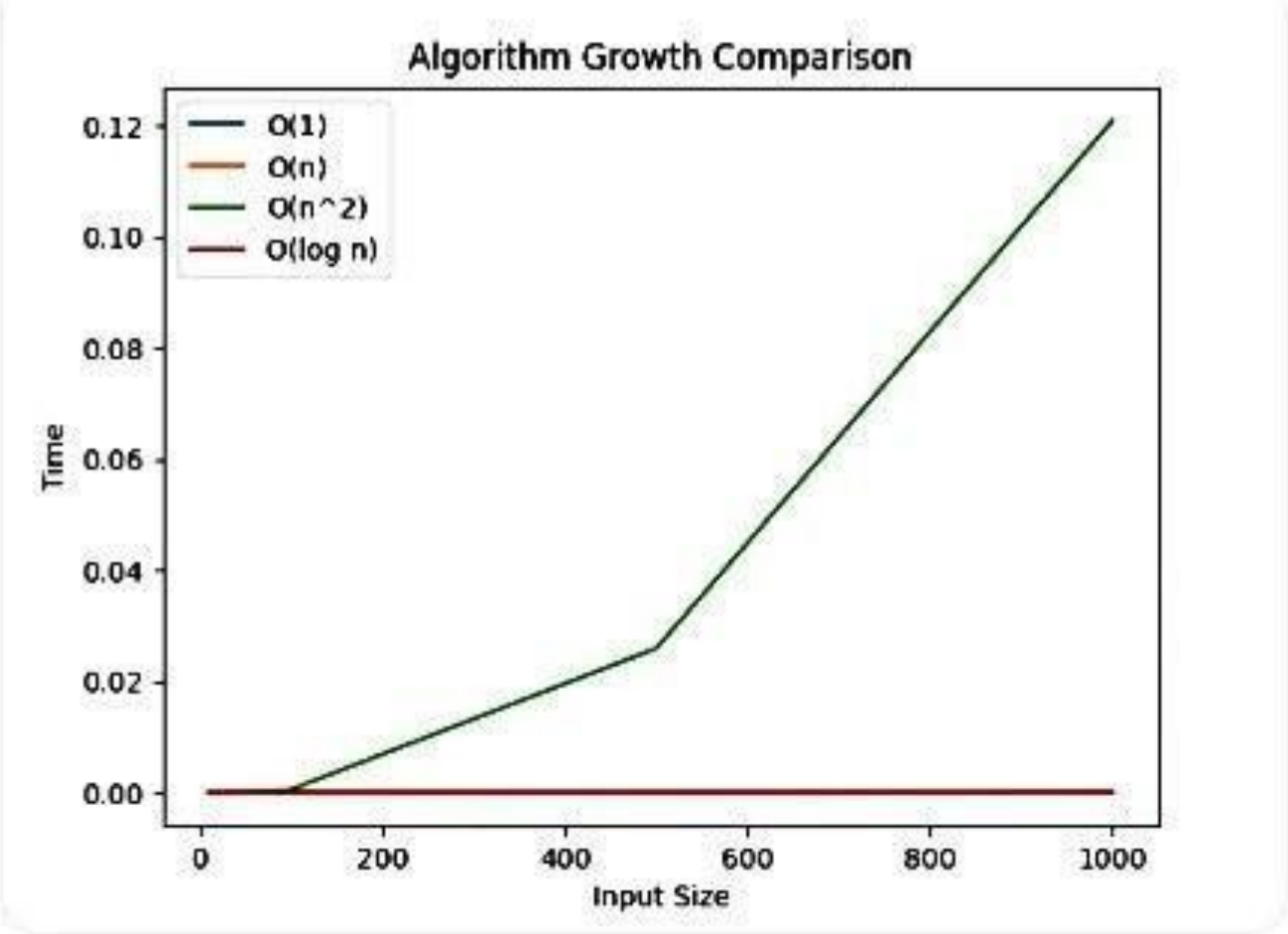
hello.py > linear_time

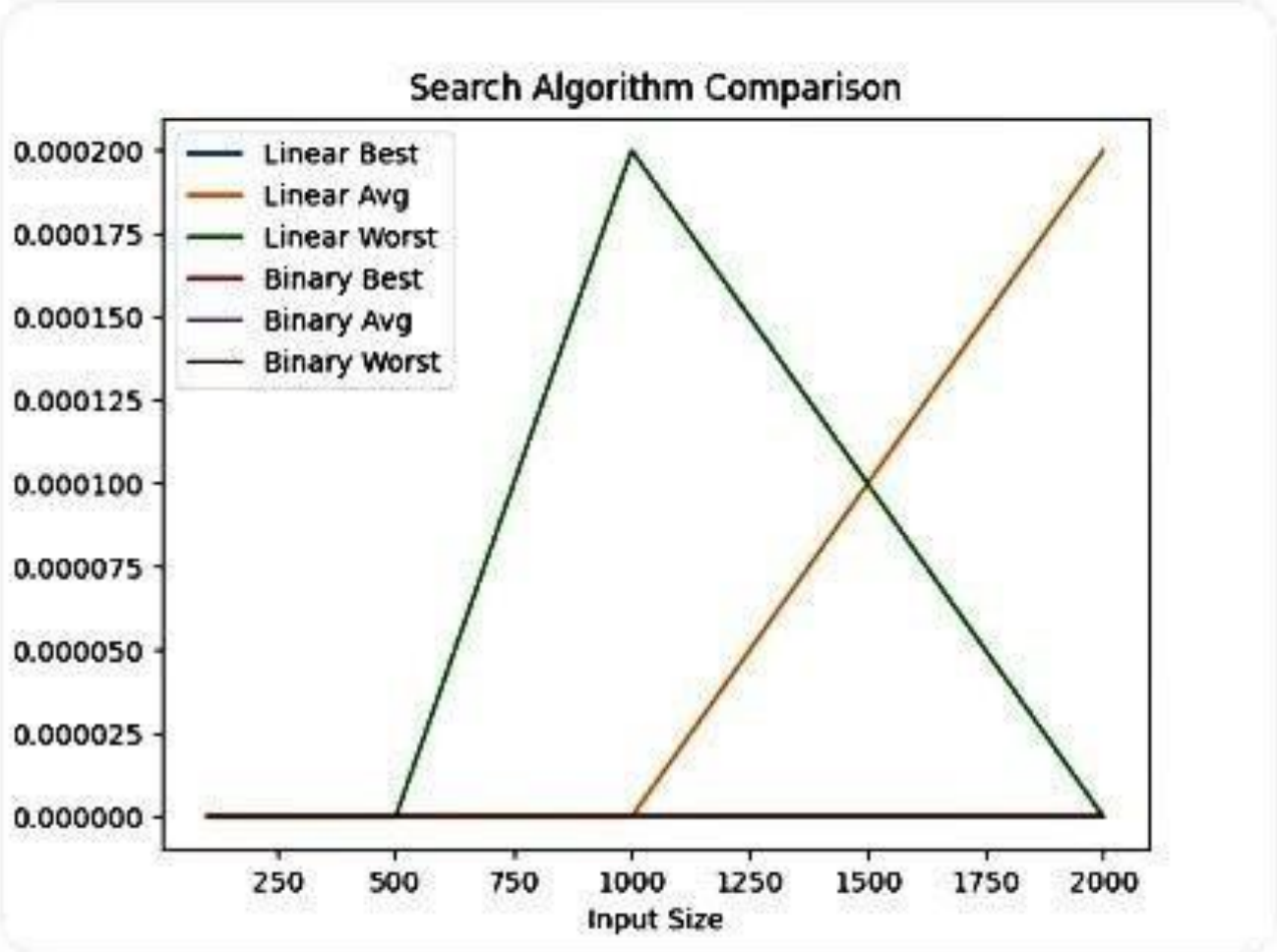
```python
    # Binary Search
    bin_best.append(measure_time(lambda x: binary_search(arr, best), n))
    bin_avg.append(measure_time(lambda x: binary_search(arr, avg), n))
    bin_worst.append(measure_time(lambda x: binary_search(arr, worst), n))


plt.figure()
plt.plot(sizes, lin_best, label="Linear Best")
plt.plot(sizes, lin_avg, label="Linear Avg")
plt.plot(sizes, lin_worst, label="Linear Worst")
plt.plot(sizes, bin_best, label="Binary Best")
plt.plot(sizes, bin_avg, label="Binary Avg")
plt.plot(sizes, bin_worst, label="Binary Worst")
plt.xlabel("Input Size")
plt.ylabel("Time")
plt.title("Search Algorithm Comparison")
plt.legend()
plt.show()

def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)

def fib_recursive(n):
    if n <= 1:
        return n
    return fib_recursive(n - 1) + fib_recursive(n - 2)

def fib_dp(n):
    dp = [0, 1]
    for i in range(2, n + 1):
        dp.append(dp[i - 1] + dp[i - 2])
    return dp[n]

sizes = [5, 10, 20, 30]

fact_times = [measure_time(factorial, n) for n in sizes]
fib_rec_times = [measure_time(fib_recursive, n) for n in sizes]
fib_dp_times = [measure_time(fib_dp, n) for n in sizes]

plt.figure()
plt.plot(sizes, fact_times, label="Factorial")
plt.plot(sizes, fib_rec_times, label="Fib Recursive")
plt.plot(sizes, fib_dp_times, label="Fib DP")
```

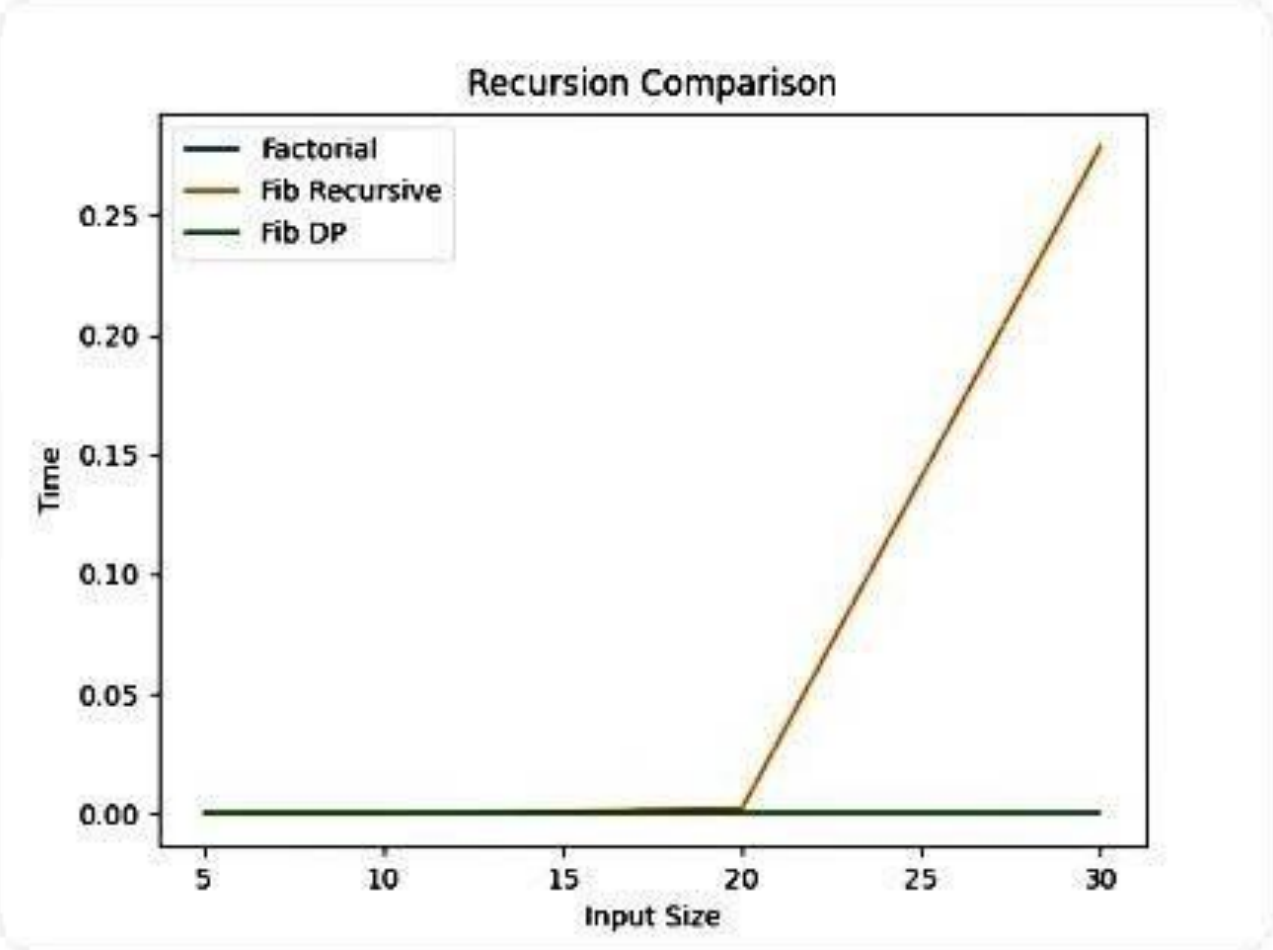
(function) def fib_recursive(n: Unknown) -> Unknown

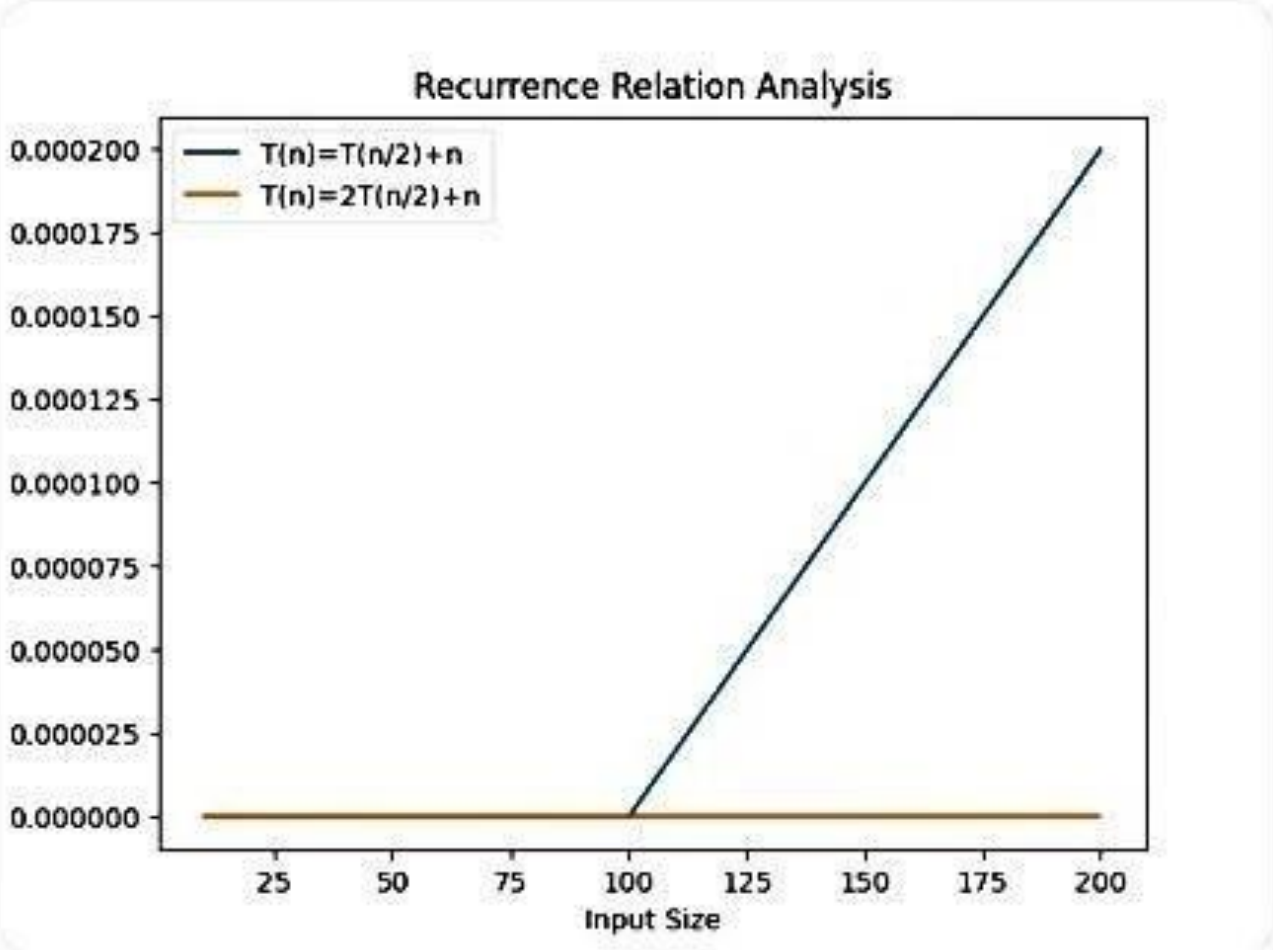
hello.py > linear_time

```python
plt.plot(sizes, fib_dp_times, label="Fib DP")
plt.xlabel("Input Size")
plt.ylabel("Time")
plt.title("Recursion Comparison")
plt.legend()
plt.show()

def recurrence1(n):
    if n <= 1:
        return 1
    return recurrence1(n // 2) + n

def recurrence2(n):
    if n <= 1:
        return 1
    return recurrence2(n // 2) + recurrence2(n // 2) + n


sizes = [10, 50, 100, 200]

rec1_times = [measure_time(recurrence1, n) for n in sizes]
rec2_times = [measure_time(recurrence2, n) for n in sizes]

plt.figure()
plt.plot(sizes, rec1_times, label="T(n)=T(n/2)+n")
plt.plot(sizes, rec2_times, label="T(n)=2T(n/2)+n")
plt.xlabel("Input Size")
plt.ylabel("Time")
plt.title("Recurrence Relation Analysis")
plt.legend()
plt.show()
```


Algorithm Growth Comparison
O(1)
O(n)
O(n^2)
O(log n)
Time
0.12
0.10
0.08
0.06
0.04
0.02
0.00
0
200
400
600
800
1000
Input Size


Search Algorithm Comparison
Linear Best
Linear Avg
Linear Worst
Binary Best
Binary Avg
Binary Worst
0.000200
0.000175
0.000150
0.000125
0.000100
0.000075
0.000050
0.000025
0.000000
250
500
750
1000
1250
1500
1750
2000
Input Size


Recursion Comparison
Factorial
Fib Recursive
Fib DP
Time
0.25
0.20
0.15
0.10
0.05
0.00
5
10
15
20
25
30
Input Size


Recurrence Relation Analysis
T(n)=T(n/2)+n
T(n)=2T(n/2)+n
0.000200
0.000175
0.000150
0.000125
0.000100
0.000075
0.000050
0.000025
0.000000
25
50
75
100
125
150
175
200
Input Size